



Universidad
Indoamérica

UNIDAD 2 · FUNDAMENTOS DE SISTEMAS OPERATIVOS

Llamadas al sistema y E/S de bajo nivel

Universidad Tecnológica Indoamérica

Laboratorio TD1

Abstracción

- Simplificar y estandarizar
- Portabilidad del núcleo y de los programas (POSIX)
- Facilitar el desarrollo de controladores
- Estabilizar el entorno de ejecución

Las abstracciones del curso

Archivos (U3) · procesos y memoria virtual (U4) · comunicación (U5, U6, U8) · hilos (U7) · y ahora agentes (U9)

Protección

- Identificación y derechos de uso y de acceso
- Aislamiento entre procesos y entre usuarios
- Cifrado de datos
- Políticas de seguridad (red, archivos...)

2026: los mismos principios se aplican a un nuevo "usuario": el **agente de IA** (identidad propia, mínimo privilegio, auditoría — U9).

Virtualización

Portabilidad y programación

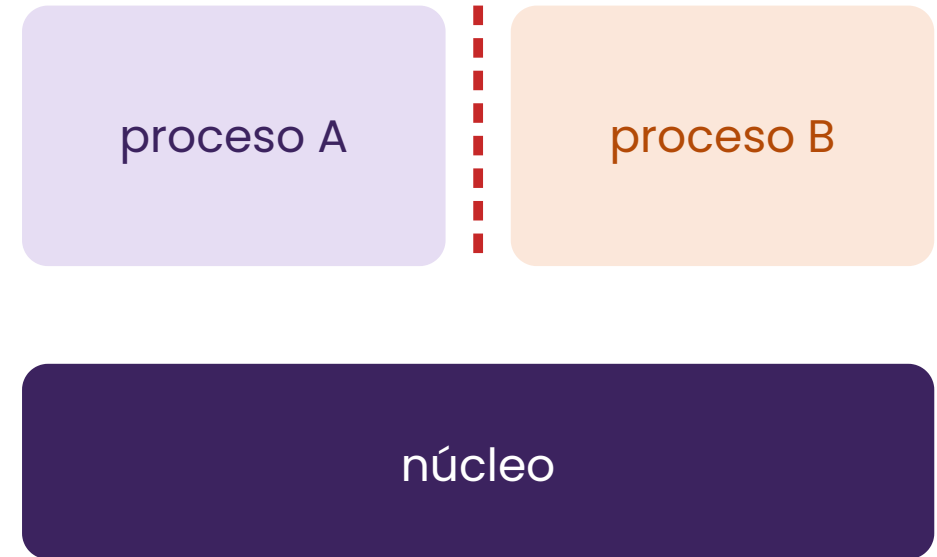
- VFS: una sola interfaz para ext4, Btrfs, NFS, /proc ...
- Una vista homogénea del hardware (discos USB, SATA y NVMe vistos igual)
- POSIX estandariza las funciones del sistema

Estabilidad y evolución

- Aislamiento de procesos: un fallo no tumba la máquina
- Simular una máquina u otro SO: QEMU/KVM, WSL 2
- Contenedores: espacios de nombres + cgroups
- IPv4 sobre IPv6

El concepto de proceso

- Cada proceso tiene su propio **contexto**
- Una vista **privada** de la memoria: no ve la de los demás
- Está aislado y **se cree solo**: los otros procesos y el propio sistema le están ocultos



Dos modos de ejecución del procesador

Modo usuario (no privilegiado)

Por defecto. El programa solo suma, lee y escribe **dentro de su propia memoria**.

Modo núcleo (privilegiado)

Para todo lo demás el proceso **pide permiso** con una instrucción especial —una llamada al sistema—: abrir, modificar o renombrar un archivo, pedir memoria, crear o terminar una tarea...

El hardware cambia de modo y el SO verifica que la operación esté permitida: todo lo que podría dañar a otras tareas o usuarios.

Modo núcleo y el núcleo

- Algunas instrucciones (gestión de memoria, interrupciones) están **prohibidas** en modo usuario
- El núcleo **gestiona** procesos, pero **no es un proceso**
- Administra recursos y acceso al hardware; siempre reside en memoria; se carga al arrancar (U1, A1.3)
- Un proceso de usuario **pasa a modo núcleo** para usar un servicio del núcleo

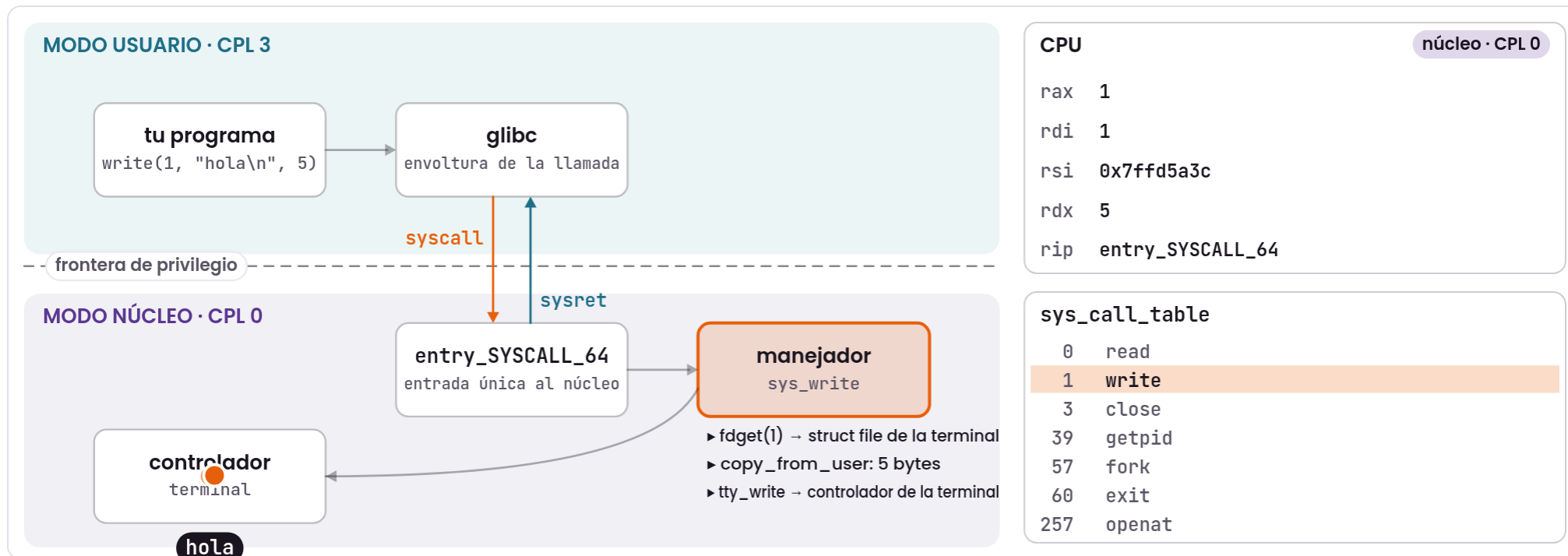
Llamadas al sistema: la interfaz

- Interfaz entre el modo usuario y el modo núcleo
- El cambio de modo lo dispara una instrucción: `ta` (Sparc), `int 0x80` (x86 antiguo), hoy `syscall` (x86-64), `svc #0` (ARM64), `ecall` (RISC-V)
- El núcleo tiene un manejador de entrada y una tabla de llamadas

¡Las llamadas son caras!

El original decía "varios cientos de ciclos". Medido en el equipo del curso: **≈ 0,7 μs por iteración** `read` + `write` (miles de ciclos).

El viaje de `write(1, "hola\n", 5)`



La instrucción, vista con objdump

REGISTRO	WRITE(1, BUF, 5)
rax	1 (número de write)
rdi	1 (descriptor)
rsi	dirección de buf
rdx	5

Salida real

```
$ objdump -d --no-show-raw-insn /usr/lib/x86_64-linux-  
gnu/libc.so.6 | grep -A6 -E '^[0-9a-f]+'  
<__write_nocancel@@'  
0000000000129060 <__write_nocancel@@GLIBC_PRIVATE>:  
129060: endbr64  
129064: mov    $0x1,%eax  
129069: syscall  
12906b: cmp    $0xffffffffffffffff,000,%rax  
129071: ja     129078  
<__write_nocancel@@GLIBC_PRIVATE+0x18>  
129073: ret
```

Errores: -1, errno y perror

- Una llamada que falla devuelve **-1** y deja el código en `errno`
- Mensaje: `void perror(const char *s);` o `%m` en `fprintf` (extensión de glibc)
- En el curso: `exit_if(rc == -1, "write")`

Salida real

```
$ ./echo blabla < . 1>&0
write: Bad file descriptor
[exit status 1]
```

C

```
ssize_t rc = write(STDOUT_FILENO, buf, n);
if (rc == -1)
    fprintf(stderr, "write: %m\n");
exit_if(rc == -1, "write");
```

Rastrear las llamadas: **strace**

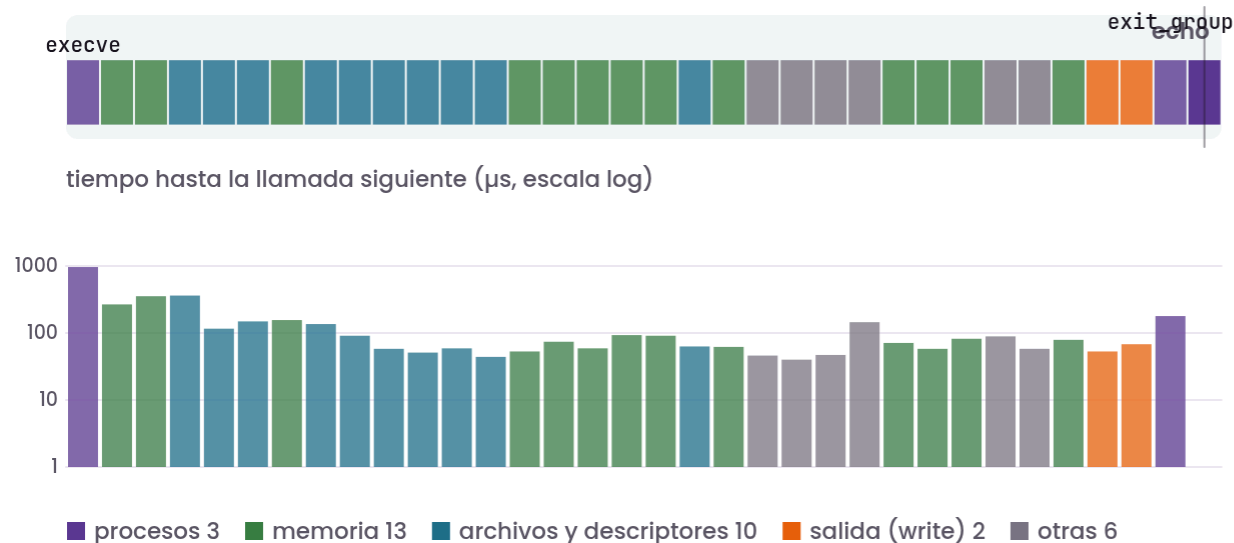
Salida real

```
$ strace -r ./echo hola > /dev/null      # the whole life of the process
0.000000 execve("./echo", [ "./echo", "hola"], 0x7ffff5a76120 /* 129 vars */) = 0
0.000965 brk(NULL)                      = 0x615cdaf80000
0.000267 mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x747beb566000
0.000354 access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
0.000362 openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
0.000116 fstat(3, {st_mode=S_IFREG|0644, st_size=108847, ...}) = 0
0.000149 mmap(NULL, 108847, PROT_READ, MAP_PRIVATE, 3, 0) = 0x747beb54b000
0.000156 close(3)                       = 0
...
```

Salida real

```
0.000089 getrandom("\x70\x5a\xe6\xac\xf7\xee\x72\x1b", 8, GRND_NONBLOCK) = 8
0.000058 munmap(0x747beb54b000, 108847) = 0
0.000079 write(1, "hola", 4)              = 4
0.000053 write(1, "\n", 1)                = 1
0.000068 exit_group(0)                  = ?
0.000179 +++ exited with 0 +++
```

La traza completa, llamada por llamada



```
set_tid_address(0x747beb548d68) = 809641
set_robust_list(0x747beb548a20, 24) = 0
rseq(0x747beb548680, 0x21, 0, 0x53053053) = 0
mprotect(0x747beb40e000, 16384, PROT_READ) = 0
mprotect(0x615caa96a000, 4096, PROT_READ) = 0
mprotect(0x747beb5ab000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, ...
getrandom("\x70\x5a\xe6\xac\xf7\xee\x72\x1b", 8, GRND_...
munmap(0x747beb54b000, 108847) = 0
write(1, "hola", 4)      = 4
write(1, "\n", 1)       = 1
exit_group(0)           = ?
+++ exited with 0 +++
```

Llamadas al sistema frente a funciones de biblioteca

Llamadas al sistema (man 2)

Inodos: `stat`, `unlink`, `chmod`, `link` ...

Descriptores: `open`, `read`, `write`, `lseek`,
`close`, `fcntl` ...

Biblioteca de C (man 3)

`fopen`, `fread`, `fprintf`, `fclose` ... **no son llamadas al sistema**: usan `FILE *` (con un búfer) en lugar de un descriptor. `fileno()` convierte uno en otro.

Leer datos: `read`

C

```
ssize_t read(int fd, void *buf, size_t count);
```

- Lee **hasta** `count` bytes del archivo de `fd` y los guarda en `buf`, que ya debe estar reservado (`count` ≤ su tamaño)
- Devuelve los bytes leídos, **0 al final del archivo** o -1 en error
- **Puede devolver menos** de lo pedido: tubería, terminal, red o una señal (`EINTR` si no leyó nada)

Escribir datos: `write`

C

```
ssize_t write(int fd, const void *buffer, size_t count);
```

- Escribe los primeros `count` bytes de `buffer` en el archivo de `fd`
- Devuelve `-1` en error o los bytes escritos

Corrección al original: "si escribe menos que `count`, es un error" no siempre es cierto. Con tuberías o sockets llenos, o una señal a mitad, la escritura puede ser **parcial y válida**: se repite con el resto (`write_all`).

¿Dónde está el error?

C

```
/* slide 51 of the original */
while ((byte = read(fd, buf, length))  $\neq$  0) {
    if (byte == -1) {
        perror("read() failed"); exit(1);
    }
    length = length - byte;
    buf = buf + length;      /* BUG */
}
```

C

```
/* fixed: advance by what was READ */
while (length > 0) {
    byte = read(fd, buf, length);
    if (byte == -1 && errno == EINTR)
        continue;
    exit_if(byte == -1, "read");
    if (byte == 0)          /* end of file */
        break;
    length -= byte;
    buf += byte;
}
```

El mismo bucle con un archivo y con una tubería

Original (diapositiva 51)

```
while ((byte = read(fd, buf, length)) != 0) {
    if (byte == -1) {
        perror("read() failed"); exit(1);
    }
    length = length - byte;
    buf = buf + length;
}
```

buf = 43 length = 0

\$ strace -e trace=read ./prog (pipe)

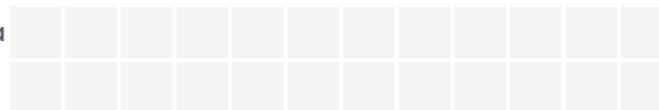
read(3, buf+36, 16) = 9

read(3, buf+43, 7) = 7

read(3, buf+43, 0) = 0

tubería: el escritor hace 4 write() de 4 + 4 + 9 + 7 bytes

tubería



memoria del proceso: char buf[24] y lo que viene después

+0 r e a d

+12 () m

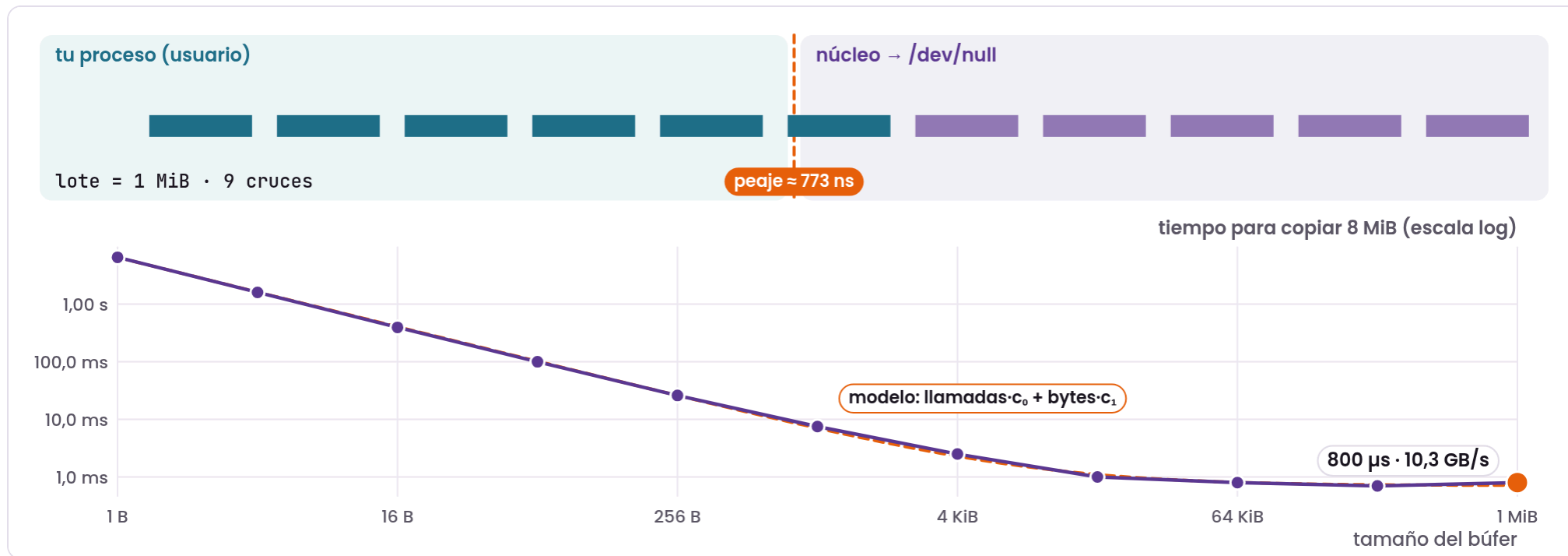
fin de buf[24] · memoria vecina: otras variables, dirección de retorno...

+24 ? ? ? ? ? ? ? ? ? ? ? ?

+36 a y r e t u l e s s

buf = 43

¿Cuánto cuesta cada cruce? Medido



Menos cruces, mismo trabajo

vDSO

`clock_gettime` sin entrar al núcleo.

io_uring

Colas compartidas:
cientos de operaciones
por llamada.

eBPF

Observar llamadas de
todo el sistema sin
detener procesos.

TD1 resuelto + desafío con IA

- Compila y ejecuta los 7 ejercicios; rompe uno a propósito
- Explica cada línea de `strace ./echo`
- Desafío con IA: medir el costo de las llamadas en tu equipo

Bash

```
make -C tds && tds/tests/run.sh td1
```



Universidad
Indoamérica

PARA LLEVAR

Tres preguntas

1. ¿Por qué el núcleo no es un proceso y por qué no es root?
2. ¿Qué devuelve `read` al final del archivo y qué hace glibc con -9?
3. ¿Qué error tenía el ejemplo de la diapositiva 51?